



FORMAL VERIFICATION

0x

AllowanceHolder

Formal Verification

FINAL REPORT

July 2026

BAILSEC.IO



Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.



1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Ox – AllowanceHolder – Formal Verification - FV Report
Website	Ox.org
Language	Solidity
Methods	Formal Verification
Certora Prover version	8.13.1
Github repository	https://github.com/OxProject/Ox-settler/tree/5a23151a10bc0a9af443dcb47eda039cf6295e01

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium						
Low						
Informational						
Governance						
Total						

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Formal Verification Methodology

Certora Formal Verification (FV) mathematically proves that a smart contract behaves correctly by checking its code against a formal specification. Unlike testing and fuzzing, which only explore particular execution paths, Certora FV reasons over every reachable state and execution path.

In practice, the desired properties are written as rules in CVL (Certora Verification Language) and submitted, together with the compiled Solidity contract, to a remote prover. The prover translates the contract's bytecode and the rules into a mathematical model and uses it to determine whether each rule holds.

Types of Properties

Properties are categorized following the [official Certora methodology](#). Valid State, Variable Transitions and State Transitions properties are **parametric** – the Prover checks them automatically against every external function in the contract, including any added after the specification is written. High-Level properties, by contrast, target specific function sequences.

Valid State – System-wide invariants that must hold at all times. They define the legitimate configurations the system's variables may occupy (e.g. accounting consistency, structural integrity) and assert that exactly one valid state holds at any moment. Once established, they act as trusted premises for other properties via `requireInvariant`, lowering verification overhead.

Variable Transitions – Properties governing how individual storage variables are permitted to evolve. Some must move monotonically across the contract's lifecycle; others may vary freely. Each records a variable's value before execution, invokes the function, and asserts the resulting change is either sanctioned or absent.

State Transitions – Properties governing movement between valid states. Extending the valid-state invariants, they enforce that states advance in the order prescribed by the state machine and only under legitimate triggers, such as specific function calls or elapsed time.

High-Level – Multi-step properties capturing end-to-end business logic. Rather than constraining one concrete element, they characterize the system as a whole from the user's perspective (for instance, solvency: value is neither created nor destroyed unintentionally). Unlike parametric properties, they validate behavior across specific function sequences.

Unit Tests – Borrowing from QA, these isolate individual functions to confirm their basic behavior: revert conditions, intended state effects, and the absence of side effects elsewhere. Each targets a single function call rather than the full external surface.

Verification Process

1. **Setup phase:** Define ghost variables, storage hooks, CVL summaries, and helper harnesses to model contract state in CVL. Establish verification harnesses and configurations for two independent targets (AllowanceHolderHarness, covering the main, context, fallback, and msgSender rule sets, and TransientStorageHarness, covering the transientStorage rule set). This phase addresses several prover limitations:
 - Transient storage model (TransientStorage.spec): AllowanceHolder keeps all allowance state in EIP-1153 transient storage, and the slot is derived from an assembly keccak256 over the packed [operator, owner, token] preimage. The Prover cannot reason about raw tload/tstore opcodes in combination with the hashed slot. The `_get/_set` primitives are summarized via CVL functions backed by a persistent ghost mapping (TSlot => uint256) allowance, and the slot derivation `_ephemeralAllowance` is replaced with an injective ghost function `ghostEphemeralSlot[operator, owner, token]` carrying an injectivity axiom – letting rules reason about per-triple allowances symbolically. The real unsummarized assembly (slot packing and tload/tstore round-trip) is verified separately against this model in the transientStorage target.
 - ERC-20 model (ERC20.spec): real ERC-20 contracts cause solver timeouts due to implementation complexity and unbounded address space. `SafeTransferLib.safeTransferFrom` is summarized by a lightweight ghost-balance model (`ghostBalances[token][from]/[to]`) with a require reproducing the insufficient-balance revert, enabling tractable consumption/solvency reasoning over a bounded account set.
 - Low-level call summaries (allowanceHolder_main.spec): the `checkCall` confused-deputy probe and the `_msgSenderSol()` calldata-tail extraction rely on gas semantics and raw calldata that the Prover cannot model. `checkCall` is summarized by `CVLcheckCall`, backed by a `ghostIsERC20[target]` mapping, and the self-forwarded sender path is summarized with a `ghostForwardedSender` ghost – preserving the behavioral property (reject if the target looks like an ERC-20; recover the appended sender) while eliminating gas/assembly complexity.
 - Fallback dispatch tracking (allowanceHolder_fallback.spec): AllowanceHolder has no Solidity function selectors — every call lands in a hand-rolled assembly fallback that routes on selector. To prove that the fallback routes to the right function, the `exec` and `transferFrom` implementations are summarized with CVL functions that flip the ghost booleans `ghostExecCalled` / `ghostTransferFromCalled`. This summaries are

used by routing raw calldata from a mock callers back into the fallback via `DISPATCH(use_fallback=true)` — so rules can assert which branch a given selector reaches and that the four branches are mutually exclusive. The summaries of the “exec” and “transferFrom” functions are only used to prove the routing by the fallback works properly. The validity of each branch is proven separately using Harness functions that allow directly calling the internal functions the fallback routes to.

- Persistent-storage hook (`allowanceHolder_main.spec`): an `ALL_SSTORE` hook scoped to `currentContract` raises a persistent ghost `wrotePersistentStorage`, declared persistent so it survives the havoc of the external call inside `exec()`. This lets rules prove the invariant that `AllowanceHolder` never writes persistent storage [all state is transient], without false positives from `SSTOREs` performed by call targets.
- Harnesses and mock targets: `AllowanceHolderHarness` exposes the internal API [`_rejectIfERC20Inner`, `_msgSender`, `exec/transferFrom` wrappers, selector getters] and overrides `_rejectIfERC20` to bypass the unmodelable `DANGEROUS_freeMemory` modifier; `TransientStorageHarness` exercises the real assembly slot construction and `tload/tstore` round-trips. The Prover models external calls to an unknown contracts by havocing — assigning arbitrary new values to all storage and ghost variables. This is sound, since the external contract might be able to change any of that state, but it wipes out the invariants a rule is trying to prove and makes the rules unprovable. To keep the otherwise unbounded external-call space tractable, the specification confines each external call to a family of mock targets [`callback`, `active/double-callback`, `const-return`, `value-accept`, `revert`, `context`, `caller`, `return-size`], wired in via link directives and bounded `DISPATCH` lists, so the Prover reasons about each mock's concrete behaviour instead.
-

2. **Crafting Properties:** Express invariants and rules in CVL, beginning with valid-state invariants [which then serve as trusted preconditions for later rules], proceeding to variable and state transitions, and culminating in high-level and unit test properties.

Project Structure

```
certora/
├── configs/
│   ├── AllowanceHolder_context.conf
│   ├── AllowanceHolder_fallback.conf
│   ├── AllowanceHolder_main.conf
│   ├── AllowanceHolder_msgSender.conf
│   └── AllowanceHolder_transientStorage.conf
├── harnesses/
│   ├── AllowanceHolderHarness.sol
│   └── TransientStorageHarness.sol
├── helpers/
│   ├── IAllowanceHolderExtended.sol
│   ├── MockActiveCallbackTarget.sol
│   ├── MockCallbackConfig.sol
│   ├── MockCallbackTarget.sol
│   ├── MockCallerContract.sol
│   ├── MockConstReturnTarget.sol
│   ├── MockContextContract.sol
│   ├── MockDoubleCallbackTarget.sol
│   ├── MockExecRevertCaller.sol
│   ├── MockReturnSizeTarget.sol
│   ├── MockRevertTarget.sol
│   ├── MockTransferFromReturnCaller.sol
│   ├── MockValueAcceptTarget.sol
│   └── MockValueAcceptTarget.sol
├── specs/
│   ├── AllowanceHolder_context.spec
│   ├── AllowanceHolder_fallback.spec
│   ├── AllowanceHolder_main.spec
│   ├── AllowanceHolder_msgSender.spec
│   ├── AllowanceHolder_transientStorage.spec
│   ├── base/
│   │   ├── allowanceHolder_context.spec
│   │   ├── allowanceHolder_fallback.spec
│   │   ├── allowanceHolder_main.spec
│   │   ├── allowanceHolder_msgSender.spec
│   │   └── allowanceHolder_transientStorage.spec
│   └── setup/
│       ├── ERC20.spec
│       └── TransientStorage.spec
```


Assumptions

Formal verification relies on assumptions about the code and its environment in order to avert prover timeouts, accommodate tool limitations, and uphold state consistency. Yet because an unsound assumption can mask genuine bugs by excising reachable states from the analysis, each assumption is classified into one of four categories:

- **Safe:** real-world constraints that leave security coverage intact
- **Proved:** preconditions that were formally verified separately
- **Trusted:** Initialization state and admin-configured parameters presumed correct, as the initialization logic lies outside the scope of verification
- **Unsafe:** Scope reductions necessitated by tractability constraints, which may inadvertently exclude valid scenarios

Safe Assumptions

These represent real-world constraints that leave security guarantees unaffected.

Environment Constraints (allowanceHolder_main.spec, setup/ERC20.spec):

- Non-payable entrypoints (transferFrom, rejectIfERC20, balanceOf) carry no value (msg.value == 0); for exec the zero-value constraint is used only to isolate target-side reverts from value-transfer reverts.
- The external caller is never the Allowance itself, except on the explicit self-forwarding branch (msg.sender != AllowanceHolderHarness / sender != currentContract) and in the AllowanceHolderContext.
- The resolved sender equals the transaction originator (sender == tx.origin), i.e. the caller is the top-level initiator rather than a forwarded or nested caller. This assumption is necessary to be able to reason about the remaining allowance which is set to 0 if "sender != tx.origin". [The complementary case (sender != tx.origin, e.g. smart-contract wallets or nested calls) is verified separately in ST-01.]

Calldata / Forwarding Constraints (allowanceHolder_msgSender.spec):

- ERC-2771 forwarding appends a 20-byte sender, so the calldata tail is at least 20 bytes long (data.length >= 20) — guaranteeing the msg.data.length - 20 slice is well-defined and does not underflow.



Address Separation [allowanceHolder_main.spec]:

- The call target is distinct from registered actors and is not itself a token (`!ghostIsERC20[target]`), so target-side execution is decoupled from the ERC-20 balance model.
- For consumption rules, owner and recipient are pairwise distinct (e.g. `recipient != msg.sender`), so balance deltas observed during a transfer are attributable to a single party.

Hashing Summary [setup/TransientStorage.spec]:

- The transient-allowance slot derivation is collision-free: distinct (operator, owner, token) triples map to distinct slots (the `ghostEphemeralSlot` injectivity axiom – the standard no-keccak-collision assumption). The real assembly preimage is separately proven equal to `abi.encodePacked(operator, owner, token)`.

ERC-20 Model [setup/ERC20.spec]:

- Token transfers conserve value: `safeTransferFrom` debits the sender and credits the recipient by the same amount over a ghost-balance mapping, and reverts on insufficient balance – the standard ERC-20 solvency invariant (total supply equals the sum of all individual balances).

Proved Assumptions

Properties that have been formally verified. They serve as foundational facts that the higher-level transition and behavioral rules rely on. Because AllowanceHolder keeps no persistent state – every allowance lives in EIP-1153 transient storage and is cleared at the end of each transaction – there is no persistent state machine to thread preconditions through, and the contract therefore does not use `requireInvariant`. The proved facts below are instead structural guarantees about deployment, storage discipline, and slot derivation.

Deployment Invariant [allowanceHolder_main.spec]:

- AllowanceHolder is either deployed at its canonical address `0x00000000000001fF3684f28c67538d4D072C22734` or running on the local test chain (`block.chainid == 31337`). This is the only CVL invariant and holds across every method (VS-02).

Storage Discipline [allowanceHolder_main.spec]:

- No persistent storage slot is ever written by AllowanceHolder under any function – all mutable state is confined to transient storage [VS-01]. This is proved parametrically over every method via an ALL_SSTORE hook scoped to the contract, and underwrites the protocol's "no passive allowances / no leftover state" design.
- The transient allowance slot changes only through exec [which sets it] and transferFrom [which decrements it]; no other entrypoint can mutate it [VT-02], and transferFrom can only decrease an allowance, never increase it [VT-01].

Slot-Derivation Soundness [allowanceHolder_transientStorage.spec]:

- The ephemeral-allowance slot is an injective function of [operator, owner, token]: distinct triples map to distinct slots, so no two allowance entries can alias [VS-03]. The production assembly preimage is additionally proved byte-for-byte equal to abi.encodePacked(operator, owner, token) [UT-25], and the summarized keccak model is proved equal to the real unsummarized assembly derivation [UT-26] – so the abstraction used by the higher-level rules faithfully represents on-chain behavior.

Unsafe Assumptions

These reduce verification scope to make the problem tractable for the prover. Where a rule restricts its own inputs for tractability, the require carries an explanatory message describing the restriction.

Function Summaries [allowanceHolder_main.spec, allowanceHolder_fallback.spec]:

- checkCall: summarized by CVLcheckCall, which returns a free per-target boolean ghostIsERC20[target] instead of executing the real staticcall + returndata/gas analysis. The confused-deputy probe outcome is therefore modeled as an oracle rather than computed – real target behavior is proven separately (excluding the gas griefing branch since the prover can not model gas)
- safeTransferFrom: replaced with a ghost-balance model. Real ERC-20 implementations [fee-on-transfer, blacklists, reentrant tokens, return-value quirks] are not modeled, so token-specific misbehavior scenarios are excluded.



Compensated Summaries [dropped in main/context/fallback, proved separately in transientStorage / msgSender]:

- Transient _get/_set are summarized by a CVL ghost mapping (raw tload/tstore not modeled); _ephemeralAllowance is summarized by the injective ghost ghostEphemeralSlot (real keccak not modeled); _msgSenderSol is summarized by ghostForwardedSender (calldata-tail extraction not modeled). Each of these is independently verified against the real assembly in the dedicated transientStorage and msgSender targets, so the summary is justified rather than assumed

Bounded External-Call Universe (allowanceHolder_main.spec, allowanceHolder_context.spec, allowanceHolder_fallback.spec):

- Unresolved external calls and callback re-entry are resolved through explicit DISPATCH lists restricted to the mock targets (callback / active-callback / double-callback / const-return / value-accept / revert / context / return-size), with default HAVOC_ECF or NONDET. Arbitrary unknown call targets and re-entrancy from contracts outside this mock set are not modeled – reentrancy coverage is limited to the modeled callback shapes.

Optimistic Prover Configuration (confs/*.conf):

- optimistic_hashing: true assumes hash injectivity (no keccak collisions)
- optimistic_fallback: true assumes the modeled fallback path for unresolved calls
- optimistic_summary_recursion: true with summary_recursion_limit: 3 bounds recursive dispatch through the summaries; deeper recursion is assumed unreachable.

Bounded Inputs (within individual rules):

- Where exhaustive reasoning over a value is intractable, rules restrict inputs with documented requires – e.g. the balanceOf returndata length is constrained to the boundary set {0, 1, 31, 32, 33} to cover the 32-byte threshold, and each forwarding rule fixes target to a single mock to avoid combinatorial dispatch blow-up.



Trusted Assumptions

Unlike protocols that gate behavior behind an initializer and admin-configured parameters, AllowanceHolder is immutable, ownerless, and stateless — there is no `initialize()`, no upgrade path, no privileged role, and no tunable parameter. Consequently there is no post-initialization state to take as a precondition. The few trusted assumptions that remain concern the deployment context and the calling convention; where a rule encodes one, the `require` carries a message describing the trusted condition.

Deployment Context (`allowanceHolder_main.spec`):

- The contract is deployed at the address `0x00000000000000001fF3684f28c67538d4D072C22734` (or, in the test environment, on local chain `block.chainid == 31337`). Integrators and the Settler contracts trust this fixed address; the `deployment-address` property is itself proved as an invariant (VS-02).

ERC-2771 Forwarding Convention (`allowanceHolder_main.spec`, `allowanceHolder_msgSender.spec`):

- On the self-call branch (`msg.sender == address(this)`), the trailing 20 bytes of `calldata` are trusted to be a sender appended honestly by the contract's own forwarding path — i.e. the rules assume `ghostForwardedSender == tx.origin` when the caller is the AllowanceHolder itself. This is the core trust boundary of the ERC-2771 context pattern: a re-entrant call carrying `msg.sender == address(this)` is assumed to have originated from the contract's documented `exec` forwarding, not from a spoofed frame. The `msgSender` rule set separately proves that, given such a frame, `_msgSender()` returns exactly that appended 20-byte tail (UT-24) and the AllowanceHolder appends the `msg.sender` properly (HL-12).
- Several rules additionally assume the entrypoint is invoked by the original transaction signer (`msg.sender == tx.origin`) on the non-self path, modeling a call made directly by the transaction's top-level account, with no distinct contract interposed between the originator and the AllowanceHolder. Note this does not assume the origin is codeless: under EIP-7702 the originating account may carry delegated code. The verification is unaffected — the contract's `tx.origin` check is a pure address comparison.

4. Verification Properties

Status indicators:  Verified /  Violated

Valid State

System-wide invariants that must hold at all times. They define the legitimate configurations the system's variables may occupy (e.g. accounting consistency, structural integrity) and assert that exactly one valid state holds at any moment. Once established, they act as trusted premises for other properties via `requireInvariant`, lowering verification overhead.

ID	Name	Description	Status	Notes
VS-01	<code>noPersistentStorageWrites</code>	No persistent storage slot is ever written; all state lives in transient storage		
VS-02	<code>deployedAtCorrectAddressOrLocalChain</code>	AllowanceHolder is either deployed at <code>0x00000000000001fF3684f28c67538d4D072C22734</code> or runs at <code>block.chainid == 31337</code>		
VS-03	<code>allowanceSlotInjectivity</code>	Distinct (operator, owner, token) triples map to distinct ephemeral slots (no aliasing)		

Variable Transitions

Properties governing how individual storage variables are permitted to evolve. Some must move monotonically across the contract's lifecycle; others may vary freely. Each records a variable's value before execution, invokes the function, and asserts the resulting change is either sanctioned or absent.

ID	Name	Description	Status	Notes
VT-01	<code>transferFromOnlyDecreases</code>	<code>transferFrom</code> strictly decreases targeted allowance, never increases the allowance		
VT-02	<code>allowanceSlotChangesOnlyViaExecOrTransferFrom</code>	Allowance slot changes only via <code>exec (set)</code> , <code>transferFrom (decrement)</code>		



State Transitions

Properties governing movement between valid states. Extending the valid-state invariants, they enforce that states advance in the order prescribed by the state machine and only under legitimate triggers, such as specific function calls or elapsed time.

ID	Name	Description	Status	Notes
ST-01	nonEoaSenderAllowanceReset	exec(): Resets the ephemeral allowance to zero after execution whenever the resolved sender differs from tx origin	✓	

High-Level

Multi-step properties capturing end-to-end business logic. Rather than constraining one concrete element, they characterize the system as a whole from the user's perspective (for instance, solvency: value is neither created nor destroyed unintentionally). Unlike parametric properties, they validate behavior across specific function sequences.

ID	Name	Description	Status	Notes
HL-01	execSetsAllowanceExactly	After exec, slot for [operator, sender, token] equals exactly amount	✓	
HL-02	execWithZeroAmountSetsZeroAllowance	exec with amount == 0 sets the ephemeral allowance to 0 and still executes the target call.	✓	
HL-03	onlyRegisteredOperatorConsumesAllowance	Only the operator can consume its allowance	✓	
HL-04	integratedNonOperatorCannotConsumeGrant	Operator isolation holds in the callback (a non-operator cannot consume the grant)	✓	
HL-05	execReturnsTargetReturndata	on a successful target call, exec does not revert and returns the target's exact returndata (length 32, word 0xbeef).	✓	
HL-06	integratedConsumptionWithinGrant	Cumulative consumed allowance is always <= granted	✓	
HL-07	transferFromMovesExactlyAmount	A successful transferFrom moves exactly amount of token from owner to recipient	✓	
HL-08	integratedRecipientInflowWithinGrant	Recipient inflow is always <= granted	✓	
HL-09	integratedFullAmountConsumable	The full granted amount is consumable (liveness)	✓	
HL-10	callRecipientRecognizesCallAsForwarded	For a exec call funneled through the AllowanceHolder, the call is recognized as forwarded	✓	
HL-11	allowanceHolderContextPassesConfusedDeputyTest	Any contract inheriting AllowanceHolderContext passes the "confused deputy" test	✓	
HL-12	callRecipientCanExtractOriginalSenderFromCallData	For a exec call funneled through the AllowanceHolder, the callRecipient can extract the original sender from the call data	✓	

Unit Tests

Borrowing from QA, these isolate individual functions to confirm their basic behavior: revert conditions, intended state effects, and the absence of side effects elsewhere. Each targets a single function call rather than the full external surface.

ID	Name	Description	Status	Notes
UT-01	rejectIfERC20FallbackTargetsCorrectAddress	Uses 0xdead when data.length <= 0x10 or extracted target <= 0xffff (precompile range)	✓	
UT-02	rejectIfERC20RevertsForERC20Target	_rejectIfERC20(): Reverts with ConfusedDeputy when the target behaves as an ERC20 token	✓	
UT-03	execPropagatesTargetRevert	exec reverts when its target call reverts (control-flow propagation only, not the revert bytes)	✓	
UT-04	allowanceKeyedOnForwardedSender	Slot written by exec is keyed on validated _msgSender, not raw msg.sender or operator	✓	
UT-05	execForwardsMsgValueToTarget	exec forwards exactly msg.value to the target	✓	
UT-06	rejectIfERC20PassesOnNonERC20Target	_rejectIfERC20 does NOT revert when the target is not an ERC20	✓	
UT-07	execRevertsForERC20Target	exec(): Reverts when the target behaves as an ERC20 token	✓	
UT-08	transferFromRevertsOnInsufficientAllowance	transferFrom(): Reverts when amount exceeds the available ephemeral allowance	✓	
UT-09	transferFromSucceedsOnZeroAmount	transferFrom(): Completes successfully without reverting when amount is zero	✓	
UT-10	transferFromDecrementsAllowanceByAmount	transferFrom(): Decreases the operator's ephemeral allowance by exactly amount	✓	
UT-11	msgSenderReturnsCallerWhenNotForwarded	_msgSender(): Returns msg sender when the caller is not the AllowanceHolder	✓	
UT-12	noEtherRetainedAfterTransferFrom	transferFrom does not affect ETH balance	✓	
UT-13	noEtherRetainedAfterExec	exec with an inert target does not retain ETH	✓	
UT-14	noEtherRetainedAfterExecWithCallback	exec with an active callback target does not retain ETH	✓	
UT-15	fallbackBranchesAreMutuallyExclusive	Fallback() The fallback's dispatch logic is mutually exclusive	✓	
UT-16	fallbackRoutesTransferFromSelector	fallback(): A call carrying the IAllowanceHolder transferFrom selector is dispatched to the transferFrom() implementation	✓	
UT-17	fallbackRoutesExecSelector	fallback(): A call carrying the IAllowanceHolder exec selector is dispatched to the exec() implementation	✓	
UT-18	fallbackRevertsOnBalanceOfSelector	fallback(): balanceOf selector path reverts	✓	
UT-19	transferFromDirtyAddressOrEthReverts	Fallback transferFrom reverts on dirty upper bits in token/owner/recipient or nonzero callvalue	✓	

UT-20	execDirtyAddressReverts	Fallback exec reverts on dirty upper 96 bits in operator/token/target	✓	
UT-21	fallbackTransferFromReturnsThirtyTwoBytesOne	fallbackTransferFromReturnsThirtyTwoBytesOne	✓	
UT-22	fallbackTransferFromReturnsTrueBool	fallbackTransferFromReturnsTrueBool	✓	
UT-23	fallbackRevertsOnUnknownSelector	fallback(): A call carrying any unrecognized selector reverts	✓	
UT-24	msgSenderReturnsLast20BytesOfMsgDataOnSelfCall	self-call _msgSender() returns the last 20 bytes of msg.data	✓	
UT-25	ephemeralPackingMatchesAbiEncodePacked	the production assembly packing (the 60 bytes fed to keccak256) is exactly // abi.encodePacked(operator, owner, token)	✓	
UT-26	realEphemeralEqualsTranscription	the real _ephemeralAllowance (unsummarized, real keccak) equals an // identical-construction transcription	✓	
UT-27	transientRoundTrip	round-trip – tload[s] after tstore[s, v] returns exactly v	✓	
UT-28	transientOverwrite	last write wins – tstore[s, v1]; tstore[s, v2]; tload[s] == v2	✓	
UT-29	transientSlotIndependence	slot independence – writing a different slot s2 does not disturb s1	✓	
UT-30	rejectIfERC20RevertsIfReturnAtLeast32Bytes	_rejectIfERC20Inner(target, data) does NOT revert when target does not respond to balanceOf with exactly 32 bytes of returndata.	✓	
UT-31	isForwardedReturnsTrueIfCallerIsAllowanceHolder	_isForwarded() returns true if the caller is the AllowanceHolder	✓	
UT-32	strippedTailEqualsMsgSender	_msgSender(): if the caller is the ALLOWANCE_HOLDER, the last 20bytes of the data are returns as the actual msg.sender	✓	
UT-33	msgSenderReturnsCallerWhenNotForwarded	_msgSender(): returns msg.sender if msg.sender is not the AllowanceHolder	✓	
UT-34	msgDataStrips20BytesIfCallerIsAllowanceHolder	_msgData strips 20 bytes from the data if msg.sender is the AllowanceHolder	✓	
UT-35	returnedMsgDataNotChanged	_msgData: returned data is not changed except for the striped msgSender if applicable	✓	



5. Real issues found

During the security engagement no real bugs were found

6. Certora Formal Verification Setup

1. In order to run the formal verification job make sure you have Certora installed properly. Follow the [official installation instructions](#).
2. To be able to run the prover using the Certora infrastructure you need a free key which you can get through the [Certora website](#). Once you have it, export it:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
```

3. Download the following repo and install it:
4. Execute the following command from your project root to run all 5 conf files:

```
certoraRun certora/confs/AllowanceHolder_context.conf  
certoraRun certora/confs/AllowanceHolder_fallback.conf  
certoraRun certora/confs/AllowanceHolder_main.conf  
certoraRun certora/confs/AllowanceHolder_msgSender.conf  
certoraRun certora/confs/AllowanceHolder_transientStorage.conf
```

5. Runs can be found on the official [certora job list page](#)

7. Required source code modifications

The Certora Prover cannot model the gas semantics of inline assembly (the GAS opcode, the EIP-150 "all but one 64th" out-of-gas heuristic), nor the `DANGEROUS_freeMemory` modifier, which manipulates the free-memory pointer directly. Several hot-path routines in `AllowanceHolder` are written in memory-safe assembly precisely to exploit these low-level behaviors, so they are opaque to verification. To work around these limitations, the affected routines were re-expressed at the Solidity level and selected functions were widened in visibility so the harness can override or summarize them. The original assembly is preserved verbatim as commented-out code, and every change is marked with `//@certora` comments. All replacements are intended to be semantically neutral with respect to the verified properties – only the gas-accounting side effects (which fall outside the spec) are dropped.

1. `CheckCall.sol` – assembly-to-Solidity replacement (lines 16, 82)
 - `checkCall` : the memory-safe assembly block performing the `staticcall` plus out-of-gas / empty-`returndata` / `extcodesize` analysis is commented out and replaced by a delegation to a new helper
 - `checkCallSolidity (new)` : reimplements the success logic in plain Solidity (`target.staticcall`, `returndata-length` and `target.code.length` checks). The gas-exhaustion branch (`invalid()` on the all-but-one-64th rule) is intentionally omitted, since gas cannot be modeled by the Prover
 - the validity of the replacement is confirmed by the test contract `CheckCallReplacementFuzzTest` located in `test/fuzz_FV/fuzztest_assembly_replacement.t.sol`
2. `AllowanceHolderBase.sol` – function splitting + visibility widening + assembly replacement (lines 22, 28, 52, 62)
 - `_rejectIfERC20` : private view `DANGEROUS_freeMemory` → internal virtual view `DANGEROUS_freeMemory`, and its body split out into `_rejectIfERC20Inner`. The outer wrapper keeps the modifier; making it internal virtual lets the harness override it to bypass `DANGEROUS_freeMemory`, which the Prover cannot model
 - `_rejectIfERC20Inner (new)` : holds the original confused-deputy check so the inner logic can be verified without the free-memory modifier; a return value (`target`) was added so the computed address is observable in rules
 - `_msgSender` : private view → internal view, to allow summarization and harness access. The inline-assembly `calldata-suffix` extraction (`shr(0x60, calldataload(sub(calldatasize(), 0x14)))`), which can not be modeled by the Prover,

is commented out and replaced by a call to a new helper holding equivalent solidity code.

- `_msgSenderSol (new)` : extracts the appended sender from the calldata tail using Solidity slicing `[address(bytes20(msg.data[msg.data.length - 20:]))]` instead of assembly. The validity of the replacement is confirmed by the test contract `MsgSenderReplacementFuzzTest` located in `test/fuzz_FV/fuzztest_assembly_replacement.t.sol`

All fuzz test were run with 10_000_000 runs to ensure the safety of the assembly code replacement

```
Ran 5 tests for test/fuzz_FV/fuzztest_assembly_replacement.t.sol:CheckCallReplacementFuzzTest
[PASS] testFuzz_CheckCall_TargetEOA(bytes,uint256) (runs: 10000000, μ: 13526, ~: 13509)
[PASS] testFuzz_CheckCall_TargetReturns32(bytes,uint256) (runs: 10000000, μ: 11600, ~: 11582)
[PASS] testFuzz_CheckCall_TargetReturnsEmpty(bytes,uint256) (runs: 10000000, μ: 11727, ~: 11709)
[PASS] testFuzz_CheckCall_TargetRevertsEmpty(bytes,uint256) (runs: 10000000, μ: 11620, ~: 11602)
[PASS] testFuzz_CheckCall_TargetRevertsWithData(bytes,uint256) (runs: 10000000, μ: 11504, ~: 11486)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 287.14s (1435.66s CPU time)
```